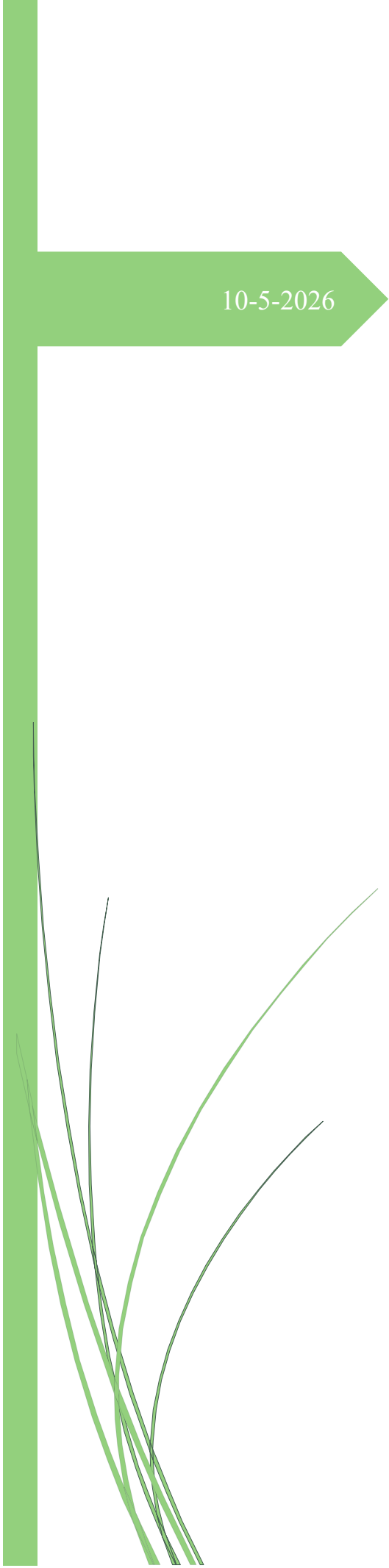


A thick green vertical bar runs down the left side of the page. A green arrow points to the right from the bar, containing the date.

10-5-2026

INFORME - PRACTICA 3

SISTEMAS OPERATIVOS

A stylized green grass graphic with several thin, curved blades extending upwards and to the right from the bottom left corner.

Marcos Encinas Lozano

INDICE

REPOSITORIO GITHUB	2
INTRODUCCION	2
OBJETIVOS	2
EXPLICACIÓN DEL PROGRAMA C++	2
1. Creación del socket.....	2
2. Configuración del servidor	3
3. Conexión con el servidor.....	3
4. Recepción de datos (bytes)	3
5. Guardado del archivo.....	3
6. Impresión de datos.....	3
GENERACIÓN DEL ARCHIVO DE RESULTADOS	4
CAPTURAS DE PANTALLA	4
CONCLUSIONES	6
INCIDENCIAS	6
EXTRA	7

REPOSITORIO GITHUB

<https://github.com/marcosEncloz/Practica-3-Marcos-Encinas-Lozano-SO>

INTRODUCCION

En esta práctica se ha desarrollado un sistema cliente-servidor utilizando Python y C++, donde el servidor envía un conjunto de datos en formato bytes y el cliente es capaz de recibirlos, almacenarlos y mostrarlos por pantalla.

El servidor, implementado en Python, genera un array de 768 valores enteros (0–255) x3 y los envía mediante un socket TCP. El cliente, implementado en C++, se conecta al servidor, recibe los datos byte a byte y los guarda en un archivo de texto.

Esta práctica permite afianzar conocimientos sobre comunicación entre procesos, sockets TCP/IP, manejo de bytes, entrada/salida en C++, modularización del código y ejecución mediante scripts en Bash.

OBJETIVOS

El objetivo principal de la práctica es implementar un sistema cliente-servidor capaz de:

- Crear un socket TCP tanto en Python como en C++.
- Establecer una conexión entre ambos procesos.
- Enviar un conjunto de datos en formato byte desde el servidor.
- Recibir los datos en el cliente sin pérdidas ni corrupción.
- Almacenar los datos recibidos en un archivo de texto.
- Mostrar los valores por pantalla.
- Automatizar la ejecución mediante un script .sh .

El objetivo educativo es reforzar el uso de sockets, comunicación TCP, lectura de bytes, modularización en C++ y automatización en Bash.

EXPLICACIÓN DEL PROGRAMA C++

1. Creación del socket

El cliente crea un socket TCP mediante:

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

Se comprueba si el socket se ha creado correctamente. En caso contrario, se muestra un mensaje de error y se finaliza la ejecución.

2. Configuración del servidor

Se configura la dirección IP y el puerto del servidor:

```
sockaddr_in serv_addr = configurarServidor("127.0.0.1", 12345);
```

El servidor se encuentra escuchando en la misma máquina, por lo que se utiliza 127.0.0.1.

3. Conexión con el servidor

El cliente intenta conectarse:

```
connect(sockfd, (struct sockaddr*)&serv_addr, sizeof(serv_addr));
```

Si el servidor está activo, la conexión se establece correctamente. Si no lo está, se muestra:

Error al conectar

4. Recepción de datos (bytes)

El cliente recibe los datos byte a byte:

```
uint8_t buffer;  
while (recv(sockfd, &buffer, sizeof(buffer), 0) > 0) {  
    datos.push_back(buffer);  
}
```

El servidor envía 768 bytes, correspondientes a tres ciclos de valores 0–255. El cliente los almacena en un vector de tipo `uint8_t`.

5. Guardado del archivo

Los datos recibidos se guardan en:

`datos_recibidos.txt`

Cada byte se escribe en una línea independiente:

```
file << (int)b << "\n";
```

6. Impresión de datos

Finalmente, el cliente muestra por pantalla todos los valores recibidos:

Bytes recibidos:

```
0  
1  
2  
...  
255
```

0
1
...

Esto permite verificar que la comunicación se ha realizado correctamente.

GENERACIÓN DEL ARCHIVO DE RESULTADOS

El archivo generado contiene los 768 valores enviados por el servidor. Este archivo permite comprobar:

- Que no se han perdido bytes.
- Que el orden es correcto.
- Que la comunicación TCP se ha realizado sin errores.

El archivo se genera automáticamente al ejecutar el cliente.

CAPTURAS DE PANTALLA

Ejecutar el servidor:

`python3 ServidorCamara.py`

```
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ cd ~/sistemas_operativos/practica_3
python3 ServidorCamara.py
Servidor escuchando en 127.0.0.1:12345
Conexión aceptada desde ('127.0.0.1', 38330)
Array de datos enviado al cliente.
```

Ejecutar el cliente:

`./cliente`

```
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ cd ~/sistemas_operativos/practica_3
./cliente
Bytes recibidos:
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
```

Practica 3 - SO

ejecutar cliente:

`./run_cliente.sh`

```
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ nano run_cliente.sh
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ chmod +x run_cliente.sh
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ ./run_cliente.sh
=====
Compilando cliente C++...
=====
Compilación exitosa.
=====
Ejecutando cliente...
=====
Error al conectar
=====
Ejecución finalizada.
Archivo generado: datos_recibidos.txt
=====
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ |
```

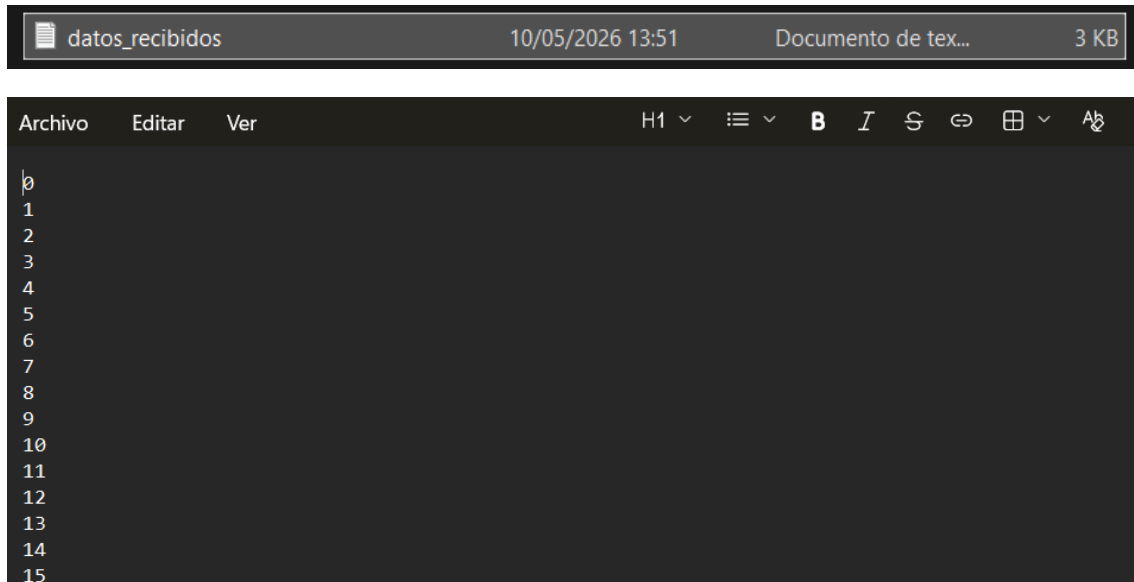
Impresión en pantalla previo a conexión

```
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ ./run_cliente.sh
=====
Compilando cliente C++...
=====
Compilación exitosa.
=====
Ejecutando cliente...
=====
Bytes recibidos:
0
1
2
3
4
5
6
7
8
9
10
250
251
252
253
254
255
=====
Ejecución finalizada.
Archivo generado: datos_recibidos.txt
=====
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ |
```

Impresión en pantalla una vez conectado

Archivo generado:

[cat datos_recibidos.txt](#)



CONCLUSIONES

En esta práctica se ha aprendido a:

- Crear y gestionar sockets TCP en Python y C++.
- Establecer comunicación entre procesos mediante red local.
- Enviar y recibir datos en formato byte.
- Modularizar un programa en C++ en funciones claras y separadas.
- Generar archivos de salida automáticamente.
- Automatizar la ejecución mediante scripts Bash.
- Resolver problemas de conexión, puertos ocupados y TIME_WAIT.

INCIDENCIAS

Durante la práctica surgieron algunas incidencias como:

- Error "Address already in use" al reiniciar el servidor demasiado rápido.
- Problemas al conectar el cliente cuando el servidor no estaba activo.
- Confusión inicial entre la IP de Windows y la IP de WSL.
- Dificultades al ejecutar el script .sh sin tener el servidor arrancado.
- Errores al guardar el archivo por permisos o rutas incorrectas.
- Necesidad de abrir dos terminales simultáneamente para cliente y servidor.

Practica 3 - SO

Todas estas incidencias fueron resueltas ajustando la configuración del servidor, liberando puertos, ejecutando procesos en terminales separadas y revisando la modularización del código.

```
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ g++ cliente.cpp -o cliente
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ ./cliente

Error al conectar
marcos@MarcosDevice:~/sistemas_operativos/practica_3$
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ ss -tln
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
LISTEN     0            1000        10.255.255.254:53        0.0.0.0:*
LISTEN     0            4096        127.0.0.54:53           0.0.0.0:*
LISTEN     0            4096        127.0.0.53%lo:53        0.0.0.0:*
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ ~/sistemas_operativos/practica_3
-bash: /home/marcos/sistemas_operativos/practica_3: Is a directory
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ python3 ServidorCamara.py
Traceback (most recent call last):
  File "/home/marcos/sistemas_operativos/practica_3/ServidorCamara.py", line 14, in <module>
    server_socket.bind((HOST, PORT))
socket.gaierror: [Errno -3] Temporary failure in name resolution
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ cd ~/sistemas_operativos/practica_3
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ python3 ServidorCamara.py
Traceback (most recent call last):
  File "/home/marcos/sistemas_operativos/practica_3/ServidorCamara.py", line 14, in <module>
    server_socket.bind((HOST, PORT))
socket.gaierror: [Errno -3] Temporary failure in name resolution
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ python3 ServidorCamara.py
Servidor escuchando en 127.0.0.1:12345
```

```
242
243
244
245
246
247
248
249
250
251
252
253
254
255
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ ss -tln
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
LISTEN     0            1000        10.255.255.254:53        0.0.0.0:*
LISTEN     0            4096        127.0.0.54:53           0.0.0.0:*
LISTEN     0            4096        127.0.0.53%lo:53        0.0.0.0:*
marcos@MarcosDevice:~/sistemas_operativos/practica_3$ |
```

EXTRA

Código .sh :

```
marcos@MarcosDevice: ~/sis  x  marcos@MarcosDevice: ~/sis  x  +  v
GNU nano 7.2 run_cliente.sh
#!/bin/bash

echo "=====
echo " Compilando cliente C++.."
echo "=====

g++ cliente.cpp -o cliente

if [ $? -ne 0 ]; then
    echo "Error: la compilación falló."
    exit 1
fi

echo "Compilación exitosa."
echo ""

echo "=====
echo " Ejecutando cliente..."
echo "=====

./cliente

echo ""
echo "=====
echo " Ejecución finalizada."
echo " Archivo generado: datos_recibidos.txt"
echo "=====

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-G Copy
```